# Contents



# 1 Terminal Management System

Flask API for the Mellon Group DevOps Bootcamp final assignment. The local stack runs with Docker Compose and includes:

- `tms-api`: Flask application built from `app/Dockerfile`
- `mysql`: official MySQL image with persistent data and automatic schema/seed import
- `redis`: official Redis image used as the cache layer

## 1.1 Project Layout

```
.
├── app/
│   ├── Dockerfile
│   ├── main.py
│   └── requirements.txt
├── db/
│   └── init/
│       ├── 01_schema.sql
│       └── 02_seed.sql
├── docker-compose.yml
├── .env.example
└── README.md
```

## 1.2 Environment

Create a local `.env` file from the template:

```
cp .env.example .env
```

Update the password values before running the stack. The real `.env` file is ignored by git.

## 1.3 Run

```
docker compose up --build
```

The API will be available at:

```
http://localhost:5000
```

Useful endpoints:

- `GET /health`: checks both MySQL and Redis connectivity and returns `503` if either component is degraded
- `GET /schema/terminals`: runs the schema exploration query from the assignment

Application logs are written to stdout with timestamp, level, and message.

## 1.4 Database Initialization

The SQL files in `db/init/` are mounted to `/docker-entrypoint-initdb.d` inside the MySQL container. The official MySQL image executes them automatically only when the database volume is created for the first time.

To recreate the database from scratch:

```
docker compose down -v
docker compose up --build
```

## 1.5 Schema Note

The `mid` field belongs to the `merchants` table. The `terminals` table stores `merchant_id`, which references `merchants.id`; therefore a terminal's MID is retrieved through a join.